**LAPORAN**

**UJIAN TEGAH SEMESTER**

**4143108 – SOFTWARE ARCHITECTURE AND DESIGN**

**Disusun Oleh :**


| NIM | NAMA |
| --- | --- |
| 11423043 | Jonathan Prima Tamba |


**SARJANA TERAPAN TEKNOLOGI REKAYASA PERANGKAT LUNAK (TRPL)**

**INSTITUT TEKNOLOGI DEL**

**2026**

**Daftar ISI**

Bab 1. PENDAHULUAN ..... 3
1.1 Latar Belakang ..... 3
1.2 Tujuan ..... 3
1.3 Cakupan Pengerjaan ..... 3
Bab 2. ISI ..... 4
2. 1 Case Study 1: MediTrack A Digital Healthcare Platform ..... 4
2.1.1. Design Thinking & Architecture Selection ..... 4
2.1.2. System Decomposition & Modeling ..... 5
2.1.3. Architecture Visualization ..... 6
2.1.4. Implementasi (Source Code Overview) ..... 7
2.1.5. Bonus Task: Performance Bottleneck & Mitigation ..... 8
2.2 Case Study 2: MediTrack Transformation ..... **Error! Bookmark not defined.**
2.2.1. Use Case Based On The Case Study ..... **Error! Bookmark not defined.**
2.2.2. Architecture Diagram ..... **Error! Bookmark not defined.**
2.2.3. Reasoning and Trade-offs ..... **Error! Bookmark not defined.**
2.2.4. Migration Strategy ..... **Error! Bookmark not defined.**
2.2.5. Implementasi (Monolith Representastion) ..... **Error! Bookmark not defined.**
2.2.6. Microservices Mapping ..... **Error! Bookmark not defined.**
Bab 3. PENUTUP ..... 9
3.1 Kesimpulan ..... 9

# Bab 1. PENDAHULUAN

## 1.1 Latar Belakang

Perkembangan teknologi informasi telah mentransformasi sektor kesehatan melalui digitalisasi layanan yang menuntut kecepatan, akurasi, dan keamanan data. MediTrack hadir sebagai platform layanan kesehatan digital yang dirancang untuk mengintegrasikan berbagai fungsi medis, mulai dari pengelolaan data pasien hingga transaksi farmasi. Dalam fase awal pengembangannya, diperlukan pemilihan arsitektur perangkat lunak yang tepat untuk menyeimbangkan kebutuhan implementasi yang cepat dengan struktur sistem yang tetap terorganisir. Pendekatan Modular Monolith dipilih karena memungkinkan pemisahan logika bisnis yang jelas antar domain kesehatan tanpa kompleksitas berlebihan pada tahap awal operasional.

## 1.2 Tujuan

Tujuan dari penyusunan laporan dan pengembangan arsitektur MediTrack ini adalah:

- Merancang struktur sistem yang memiliki pemisahan tanggung jawab (Separation of Concerns) yang jelas antar fungsi layanan kesehatan.
- Memastikan keamanan dan integritas data medis pasien melalui penerapan prinsip Security by Design.
- Membangun fondasi arsitektur yang mendukung pemeliharaan (maintainability) jangka panjang serta fleksibilitas untuk pengembangan fitur di masa depan (extensibility).
- Menyiapkan jalur evolusi sistem agar siap ditransformasikan menjadi arsitektur Microservices seiring dengan pertumbuhan beban pengguna.

## 1.3 Cakupan Pengerjaan

Pengerjaan laporan ini mencakup beberapa aspek utama, antara lain:

- Analisis Arsitektur: Melakukan justifikasi pemilihan arsitektur Modular Monolith, identifikasi trade-offs, serta analisis skalabilitas sistem.
- Pemodelan Sistem: Melakukan dekomposisi sistem menjadi modul-modul utama seperti User Management, Appointment, EHR, Pharmacy, Payment, dan Analytics.
- Visualisasi Desain: Menyusun diagram UML dan diagram arsitektur tingkat tinggi untuk menggambarkan aliran data dan titik integrasi eksternal.
- Representasi Implementasi: Memberikan gambaran struktur kode dan pemetaan modul menuju potensi transformasi Microservices di masa depan.

# Bab 2. ISI

## 2. 1 Case Study 1: MediTrack  A Digital Healthcare Platform

### 2.1.1. Design Thinking & Architecture Selection

Dalam pengembangan platform MediTrack, dipilih pendekatan Monolithic Architecture dengan konsep Modular Monolith menggunakan framework Laravel (PHP). Pendekatan ini dipilih untuk menyeimbangkan kebutuhan pengembangan cepat, kemudahan pemeliharaan, serta kesiapan terhadap pertumbuhan sistem di masa depan.

a. Justify the choice with at least three architectural principles

- Separation of Concerns (SoC): Setiap fungsi bisnis utama (User, Appointment, EHR, dll.) dipisahkan ke dalam modul-modul logis yang independen. Ini memastikan bahwa logika bisnis kesehatan tidak tercampur dengan logika pembayaran atau autentikasi.
- High Cohesion & Low Coupling: Setiap modul memiliki tanggung jawab yang spesifik dan fokus (High Cohesion). Komunikasi antar modul dibatasi melalui Service Layer atau Internal API, sehingga perubahan di satu modul tidak merusak modul lainnya (Low Coupling).
- Security by Design: Mengingat sensitivitas data medis, keamanan diintegrasikan langsung ke dalam struktur arsitektur melalui mekanisme autentikasi terpusat dan otorisasi berbasis peran (RBAC) pada setiap akses modul.

b. Identify trade-offs and limitations

- Scalability Constraints: Seluruh aplikasi harus diskalakan bersamaan (vertical/horizontal scaling pada satu unit), yang bisa kurang efisien jika hanya modul "Appointment" yang sedang sibuk.
- Deployment Risk: Kesalahan kecil pada satu modul mengharuskan seluruh sistem di-deploy ulang, yang berpotensi menyebabkan downtime pada seluruh layanan jika tidak diuji dengan ketat.
- Codebase Complexity: Seiring bertambahnya fitur, ukuran kode akan membengkak. Jika tim pengembang menjadi sangat besar, koordinasi dalam satu repository bisa menjadi tantangan.

c. Suggest how the architecture supports scalability, maintainability, and extensibility

- Scalability: Mendukung penggunaan Load Balancer dan caching (Redis) untuk performa. Tugas berat seperti pengiriman notifikasi dipisahkan ke dalam asynchronous queues.
- Maintainability: Pola Service Layer dan Repository Pattern membuat kode tetap bersih, mudah diuji (unit testing), dan mudah dipahami oleh developer baru karena batas modul yang jelas.
- Extensibility: Fitur baru dapat ditambahkan sebagai modul baru tanpa mengganggu modul lama. Struktur ini juga memudahkan evolusi menuju Microservices di masa depan.

**2.1.2.** System Decomposition & Modeling

a. List the modules/services and their responsibilities.

- User Management: Autentikasi, otorisasi, dan manajemen profil (Pasien, Dokter, Apoteker).
- Appointment: Manajemen jadwal konsultasi dan pemesanan janji temu.
- Electronic Health Records (EHR): Pengelolaan data medis sensitif, riwayat penyakit, dan hasil lab.
- Pharmacy: Manajemen stok obat dan pemrosesan resep digital.
- Payment: Penanganan transaksi pembayaran dan klaim asuransi.
- Analytics: Pengolahan data menjadi laporan performa dan tren kesehatan.

b. For each module, define key classes/entities and their relationships.

Setiap modul dalam MediTrack memiliki entitas utama yang merepresentasikan objek dalam sistem serta relasi antar entitas tersebut.

Pada modul User Management, entitas utama terdiri dari User, Role, dan Profile. Setiap user memiliki satu role, dan setiap user memiliki satu profil yang berisi informasi detail sesuai jenis pengguna.

Pada modul Appointment, terdapat entitas Appointment yang berelasi dengan Patient dan Doctor. Satu pasien dapat memiliki banyak appointment, dan satu dokter juga dapat menangani banyak appointment.

Pada modul EHR, entitas utama meliputi MedicalRecord, Prescription, dan LabResult. Setiap medical record dimiliki oleh satu pasien dan dapat memiliki banyak prescription serta hasil lab.

Pada modul Pharmacy, terdapat entitas Drug, Inventory, dan Order. Prescription dari modul EHR akan menghasilkan order yang diproses oleh pharmacy. Inventory digunakan untuk mengelola stok obat.

Pada modul Payment, entitas seperti Transaction dan InsuranceClaim digunakan untuk mencatat pembayaran yang dilakukan pasien serta klaim yang diajukan ke pihak asuransi.

Pada modul Analytics, entitas seperti Report dan Metrics digunakan untuk mengolah dan menyajikan data dari berbagai modul lainnya.

Relasi antar modul tetap dijaga seminimal mungkin dengan komunikasi melalui service layer, sehingga tidak terjadi ketergantungan langsung yang berlebihan.

c. Use UML Class Diagrams or component diagrams to illustrate the design.

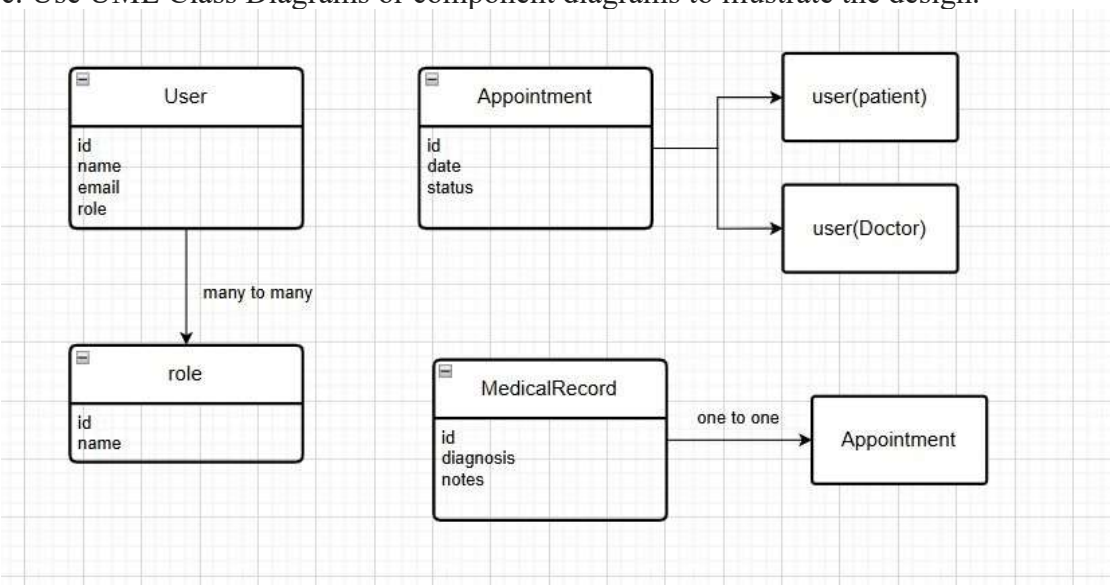



Diagram Kelas UML tersebut menggambarkan struktur data MediTrack di mana entitas User memiliki relasi many-to-many dengan role untuk manajemen akses , sementara entitas Appointment menghubungkan pasien dan dokter yang kemudian menghasilkan satu MedicalRecord melalui relasi one-to-one untuk mencatat hasil diagnosis , serta diperluas dengan modul Pharmacy yang mengelola resep obat melalui entitas Prescription dan Order

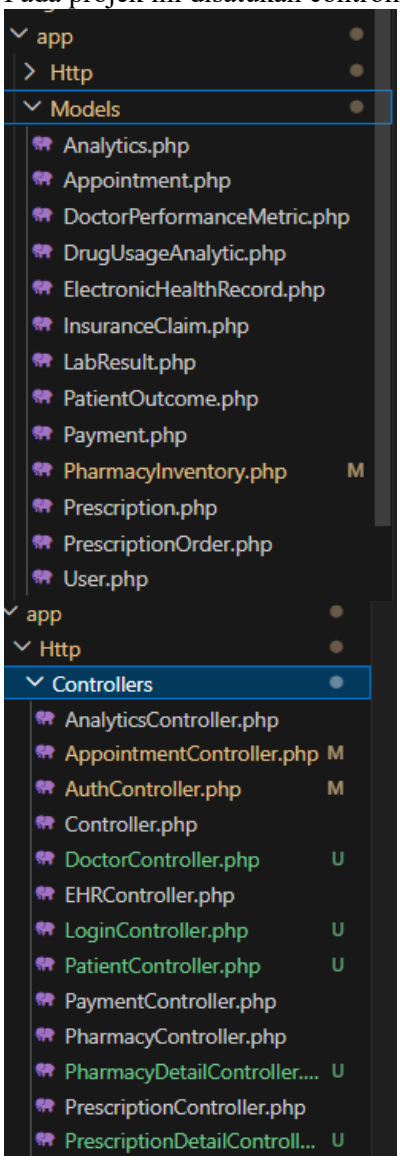
### 2.1.3. Architecture Visualization

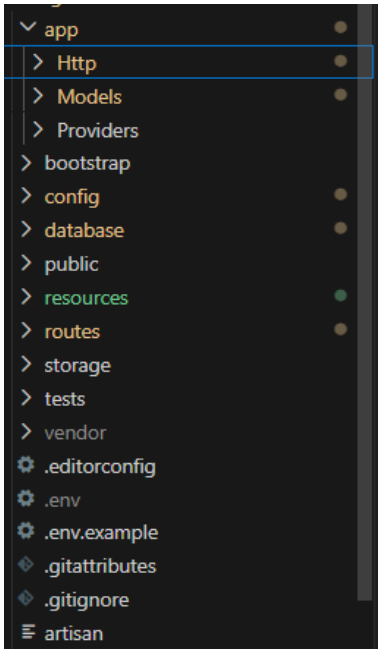
Arsitektur ini memvisualisasikan bagaimana MediTrack beroperasi sebagai satu kesatuan sistem yang terorganisir:

- Boundary & Layers: Sistem dibagi menjadi lapisan Presentation untuk menangani request pengguna, Service Layer sebagai pusat logika bisnis dari setiap modul (User, Appointment, EHR, Pharmacy, Payment, Analytics), dan Data Access Layer yang berinteraksi dengan satu database bersama.
- Data Flow & Integration: Aliran data dimulai dari API Gateway menuju modul spesifik, di mana komunikasi antar modul (misalnya modul Appointment memvalidasi jadwal ke modul User) dilakukan secara internal melalui service calls untuk menjaga keteraturan sistem.
- External Systems Interconnectivity: Diagram ini menunjukkan titik integrasi pada modul Payment yang terhubung dengan payment gateway eksternal dan modul Pharmacy yang berinteraksi dengan API pihak ketiga untuk sinkronisasi stok obat secara otomatis.

### 2.1.4. Implementasi (Source Code Overview)

Pada projek ini disatukan controller,database,dan model nya dalam satu folder.






Database 1 untuk semua fitur

### 2.1.5. Bonus Task: Performance Bottleneck & Mitigation

# Bab 3. PENUTUP

## 3.1 Kesimpulan

Berdasarkan analisis dan perancangan yang telah dilakukan, dapat disimpulkan bahwa:

- Pemilihan Arsitektur: Penggunaan Modular Monolith merupakan solusi yang paling tepat untuk fase awal MediTrack karena berhasil menyeimbangkan prinsip Separation of Concerns dengan efisiensi pengembangan. Struktur ini memungkinkan pengelolaan data medis yang sensitif tetap aman di bawah prinsip Security by Design.
- Struktur Sistem: Dekomposisi sistem ke dalam enam modul utama (User Management, Appointment, EHR, Pharmacy, Payment, dan Analytics) memberikan batasan tanggung jawab yang jelas, sehingga memudahkan proses pemeliharaan dan pelacakan bug.
- Skalabilitas dan Evolusi: Meskipun memiliki batasan dalam skalabilitas independen, desain ini telah menyiapkan jalur migrasi yang jelas menuju arsitektur Microservices melalui pemetaan layanan yang terdefinisi dengan baik tanpa mengganggu fungsionalitas yang sudah ada.
- Interaktivitas Data: Melalui pemodelan UML, terlihat bahwa alur data dari pendaftaran pengguna, penjadwalan janji temu, hingga penerbitan resep medis dan pembayaran telah terintegrasi secara logis dan siap untuk diimplementasikan ke dalam kode program.